

NZFIK: A Fast Method for Human Pose Estimation

Yu An^a, Wenbo Liu^b, Guowei Wang^{*}, Jiayuan Sun^c, Ti Wang^d

North China Institute of Computing Technology, Beijing 100083, China

^a936092522@qq.com, ^blwbbupt@foxmail.com, ^{*} hugowangguowei@hotmail.com, ^cjiayuan_sun@163.com, ^dmaimaiti@hotmail.com

Abstract. This article proposes a non-iterative zero-error fast inverse kinematics algorithm (NZFIK) based on the follow the leader (FTL) algorithm for human motion restoration with a small amount of sensor data. It can calculate joint positions point by point with the strategy of minimizing root pose changes. The algorithm can quickly calculate typical joint connection forms such as chain and closed-loop, and compared with the forward and backward reaching inverse kinematics (FABRIK) algorithm, it significantly improves computational efficiency while obtaining approximate results. Finally, this article takes the rigid connection solution of human upper limb joints as an example to validate the effectiveness of the algorithm. The results show that the NZFIK algorithm has good performance in restoring human upper limb posture based on a small number of end effectors on the wrist and head.

Keywords: Inverse kinematics, Human pose estimation, End-effector, Motion capture

I. INTRODUCTION

As a fundamental support technology for human-computer interaction, simulation, game production, and other fields, human pose estimation^[1] technology has gradually become a research hotspot with the rapid expansion of application requirements. In the vast majority of application scenarios, the main goal of human motion restoration technology is to accurately and completely output human posture through calculation, while accessing partial or imprecise human joint information. In response to this goal, predecessors have proposed various solutions that can to some extent solve the problem of action restoration. With the rapid expansion of the application field of human motion restoration, this technology can bring value to desktop, mobile, or wearable devices. However, there are significant differences in computing power and resource disposal permissions between these application modes. Therefore, it is of great value to find a fast, convenient, and precise method for restoring human movements.

Inverse kinematics (IK)^[2] is the key research content of human motion restoration, which is a method to solve the motion state of each joint according to the user's given constraints and target point positions. Generally, it can be divided into three categories: Jacobian matrix method, Newton method, and heuristic method. The Jacobi matrix method is a Linear approximation of the position and orientation of the end effector to find the IK problem by constantly approaching the target position and orientation. However, the existence of singular values will cause vibration and discontinuity in the motion. The Newton method uses a second-order Taylor expansion based on the objective function to obtain a second-order approximate solution, which can return smooth motion without interruption. The joint constraints are simple, but there are problems with complex algorithms, difficulty in implementation, and high iterative calculation costs. The

representative of the heuristic iterative algorithm is the cyclic coordinate descent (CCD) and the FABRIK algorithm.

CCD provides a numerically stable solution that implements algorithms for human figure data in many computer graphics and robot applications, but there are often problems with discontinuous and oscillating movements. The FABRIK algorithm is easy to implement and can flexibly adapt to different problems, maintaining low computational overhead while handling multiple end effectors and closed-loop problems.

This article proposes a non-iterative zero-error fast inverse kinematics problem solving method based on the FTL algorithm used for rope simulation. Under the premise of naturally stable results, the joint pose can be directly obtained without iteration. The experimental results show that the algorithm proposed in this paper can be applied to most application scenarios of the FABRIK algorithm, significantly improving the computational efficiency compared to the FABRIK algorithm. In the case where the FABRIK algorithm is not applicable, it can also ensure the smoothness of joint motion and have a good display effect.

II. RELATED WORKS

The numerical solution methods are divided into Jacobian method, Newton method, and heuristic method. FABRIK^[3] is a type of heuristic algorithm introduced by Aristou and Lasenby. The concept of this algorithm is similar to the FTL algorithm, which does not use angle rotation, but updates the new position of the node along a line to the next joint. Unlike FTL running in a single iteration, FABRIK works in a forward and backward iteration mode, minimizing the distance between the target and the end effector each time. The FABRIK^[4] algorithm can solve the motion restoration problem of two typical structures: chain joint structure and closed-loop joint structure.

Currently, there are also many extensions based on FABRIK. Moya and Colloud^[5] demonstrated that FABRIK can handle target priorities and adjust the initial algorithm to handle joints with more than two parts. Aristidou^[6] applies a constrained version of the algorithm to hierarchy and priority order to control the motion of humanoid models. Agrawal and van de Panne^[7] used a data-driven priorities at the beginning of FABRIK to generate more natural human postures, while Tao and Yang^[8] extended FABRIK to handle collision free tasks. Lansley^[9] recently introduced CALIKO, an open library for the FABRIK algorithm.

However, the FABRIK algorithm still has certain problems: unsuitable motion models include precision zero error motion models, multi node chain models, and closed-loop models exceeding 3 points. For precise zero error motion models: FABRIK has the problem of motion errors caused by iteration. For the multi node chain model: when the number of nodes increases, the number of iterations of the FABRIK algorithm

increases exponentially and cannot converge quickly. For closed-loop models with more than 3 points: Using the FABRIK closed-loop algorithm can cause significant deformation.

III. NZFIK ALGORITHM

This paper proposes a new heuristic algorithm, Non iterative Zero error fast IK (NZFIK). NZFIK does not use iteration, but instead calculates each joint position one by one by locating points online, generating visually realistic poses at a lower computational cost. The main goal of the algorithm is to improve the efficiency of solving IK problems while ensuring the natural stability of the calculation results, and to make adaptive improvements to joint connection forms such as chain and closed-loop. All rigid joint models of the human body can be regarded as a set composed of a single chain, multi chain, closed loop and other structures. Therefore, the application of this algorithm can systematically solve the typical problems in Mannequin.

A. Single End-effector

Under the condition of a single end-effector, the algorithm ensures algorithm accuracy by determining the motion state of its parent node through the target position of the end effector and the position of its grandfather node, thereby moving as few joint points as possible to complete the IK problem solving.

Let the single chain be composed of n joints connected, and the set of space coordinates of each joint is marked as $\mathbf{P}$, $\mathbf{P} = \{\mathbf{P}_i = (x_i, y_i, z_i), i = 1, 2, \dots, n\}$. Starting from the root node, the set of distances between each joint and its child joints is denoted as D , $D = \{D_i = |\mathbf{P}_i - \mathbf{P}_{i+1}|, i = 1, 2, \dots, n-1\}$. The spatial coordinates of the moving target point are denoted as $\mathbf{T}$. The joint length from the last node to the preceding node is denoted as L , $L = \{L_i = D_{n-1} + D_{n-2} + \dots + D_{n-i}, i = 1, 2, \dots, n-1\}$.

Step 1: First, record the distance between the first node $\mathbf{P}_1$ and the target point $\mathbf{T}$ as $D_{(1)(T)}$, and then determine the size relationship between $D_{(1)(T)}$ and the length from joint point $\mathbf{P}_1$ to final joint point $\mathbf{P}_n$ as L_{n-1} . In the final state, $\mathbf{P}_n$ coincides with the coordinates of the target point $\mathbf{T}$, that is $\mathbf{P}'_n = \mathbf{T}$. If $D_{(1)(T)}$ is greater than L_{n-1} , it is considered that the first node must undergo a change in motion state in order for the chain to complete the motion process. At this time, according to the rope model, all other nodes on the single chain have coordinates located on the line between $\mathbf{P}_1$ and $\mathbf{T}$, as shown in formula (1).

$$\mathbf{P}'_i = \frac{L_{n-i}}{|\mathbf{T} - \mathbf{P}_i|} \mathbf{P}_1 + \left(1 - \frac{L_{n-i}}{|\mathbf{T} - \mathbf{P}_i|}\right) * \mathbf{T} \quad (i = 1, 2, \dots, n-1) \quad (1)$$

Step 2: If $D_{(1)(T)}$ is less than or equal to L_{n-1} , it is considered unnecessary for the first node to change its motion state, and the chain can complete the motion process. At this point, mark the joint point where the current position needs to be calculated as $\mathbf{P}_i$, the distance from joint point $\mathbf{P}_{i-1}$ to target point $\mathbf{T}$ as $D_{(i-1)(T)}$, and the length of joint point $\mathbf{P}_{i-1}$ to final joint point $\mathbf{P}_n$ as L_{n-i} . Starting from the previous node $\mathbf{P}_{n-1}$ of the end joint point, calculate the positions of each preceding node in sequence, that is, in the initial state, $i =$

$n-1$. If $D_{(i-1)(T)}$ is less than or equal to L_{n-i} , as shown in Figure 1, the new position $\mathbf{P}'_i$ of joint point $\mathbf{P}_i$ can be directly calculated, as shown in formula (2).

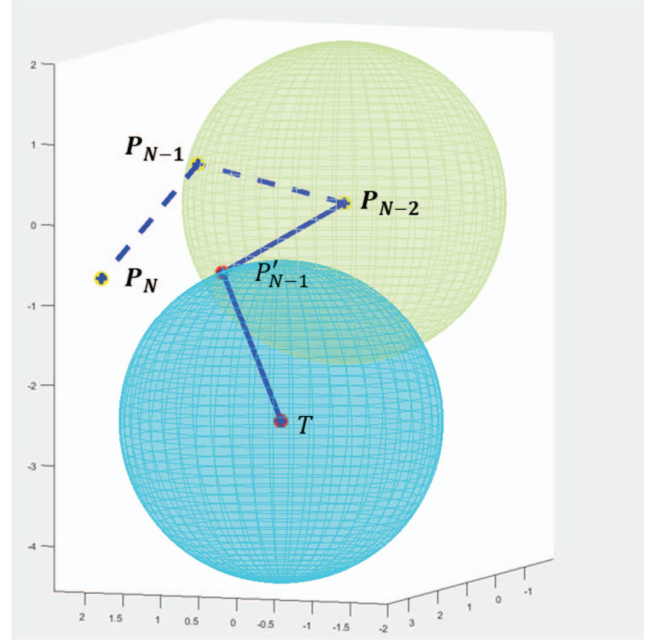


Figure 1 Schematic diagram of NZFIK algorithm under single end-effector conditions

$$\mathbf{P}'_i = \mathbf{P}_M + \frac{\sqrt{L_{n-i}^2 - D_{(M)(N)}^2}}{D_{(M)(N)}} (\mathbf{P}_N - \mathbf{P}_M) \quad (2)$$

Among them, $\mathbf{P}_M$ is the centroid of the $\mathbf{P}'_i$ motion trajectory, $\mathbf{P}_N$ is the projection point of $\mathbf{P}_i$ on the plane where the $\mathbf{P}'_i$ motion trajectory is located, $D_{(M)(N)}$ is the distance between $\mathbf{P}_M$ and $\mathbf{P}_N$, and $D_{(M)(T)}$ is the distance between $\mathbf{P}_M$ and $\mathbf{T}$.

$$\mathbf{P}_M = \mathbf{T} + \frac{L_{n-i}^2 - D_{N-1}^2 + D_{(i-1)(T)}^2}{D_{(M)(N)}} (\mathbf{P}_{i-1} - \mathbf{T}) \quad (3)$$

$$\mathbf{P}_N = \mathbf{P}_i - \frac{\mathbf{P}_i \cdot \mathbf{Q} + |\mathbf{T}|^2 - |\mathbf{P}_{i-1}|^2 + L_{n-i}^2 - D_{N-1}^2}{|\mathbf{Q}|} \mathbf{Q} \quad (4)$$

$$\mathbf{Q} = -2(\mathbf{T} - \mathbf{P}_{i-1})\mathbf{I}_3 \quad (5)$$

Where $\mathbf{Q}$ is the vector, $|\mathbf{T}|$ is the module length of target node $\mathbf{T}$, $|\mathbf{P}_{i-1}|$ is the module length of joint point $\mathbf{P}_{i-1}$, $|\mathbf{Q}|$ is the module length of vector $\mathbf{Q}$, and $\mathbf{I}_3$ is the Identity matrix of 3×3 . All preceding nodes of joint point $\mathbf{P}_i$ remain in their original motion state.

Step 3: If $D_{(i-1)(T)}$ is greater than L_{n-i} , reduce the value of i by 1, that is, move the node to be calculated forward by one joint, and repeat this process until $D_{(i-1)(T)}$ is less than or equal to L_{n-i} . At this point, perform the calculation process of step 2, and calculate the new positions of the remaining post nodes from joint point $\mathbf{P}_i$ to the rear according to the algorithm of step 2.

B. Multi End-effector

Under multi end-effector conditions, this paper proposes a dual master sub chain algorithm. The algorithm first performs the NZFIK single chain algorithm on all sub chains, and then determines the two chains with the highest weight as dual master sub chains based on the weight of each chain. Finally, based on the root node position of the dual master sub chain, the root node position is determined. Then, with the new root node position as the target point, the NZFIK algorithm is applied to all chains to solve the IK problem. The schematic diagram of the algorithm under multi chain conditions is shown in Figure 2: the chain where the solid point is located is the main sub chain.

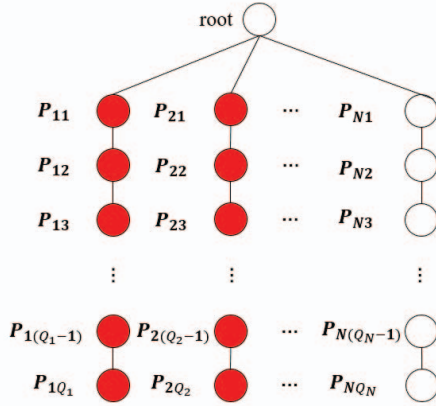


Figure 2 Schematic diagram of NZFIK algorithm under multi chain conditions

The algorithm requires each sub chain number to be given, denoted by the number n , the initial position $\mathbf{r} = (x_r, y_r, z_r)$ of the root node, the weight of each sub chain $W = \{W_i, i = 1, 2, 3, \dots, n\}$, the number of nodes $Q = \{Q_i, i = 1, 2, 3, \dots, n\}$ on each sub joint chain, the set of joint point positions $\mathbf{P} = \{\mathbf{P}_{ij}, i = 1, 2, 3, \dots, n, j = Q_1, Q_2, Q_3, \dots, Q_n\}$ on each sub chain, the set of target point positions $\mathbf{T} = \{\mathbf{T}_i = (x_{Ti}, y_{Ti}, z_{Ti}), i = 1, 2, 3, \dots, n\}$, and the set of length $L = \{D_{ij} = |\mathbf{P}_{ij} - \mathbf{P}_{i(j+1)}|, i = 1, 2, 3, \dots, n, j = Q_1, Q_2, Q_3, \dots, Q_n\}$ between connected joints. The algorithm process is as follows:

(1) When $n = 2$, first calculate the two sub chains using the single chain algorithm to obtain the two root nodes $\mathbf{r}_1$ and $\mathbf{r}_2$. If $\mathbf{r}_1 = \mathbf{r}_2$, complete the IK problem solving; If $\mathbf{r}_1 \neq \mathbf{r}_2$, then use $\mathbf{P}_{11}$ and $\mathbf{P}_{21}$ as the center of the ball and D_{11} and D_{21} as the radius to create the ball. For $\mathbf{P}_{11}$, $\mathbf{r}$, and $\mathbf{P}_{21}$, calculate the new position $\mathbf{r}'$ of $\mathbf{r}$ using the method in step 2 of 3.1, while keeping the other point positions unchanged, and complete the IK problem solution.

(2) When $n > 2$, first determine the weight size between the sub chains and obtain the two main sub chains with the highest weight, e and f . Then, perform a single chain algorithm on all sub chains, and obtain the root nodes of the two main sub chains, denoted as $\mathbf{r}_e$ and $\mathbf{r}_f$. If $\mathbf{r}_e = \mathbf{r}_f$, then $\mathbf{r}' = \mathbf{r}_e = \mathbf{r}_f$ with $\mathbf{r}'$ as the target point, reverse execute the single chain algorithm on the remaining sub chains except for the main

sub chain to complete the IK problem solution. If $\mathbf{r}_e \neq \mathbf{r}_f$, then $\mathbf{r} = (\mathbf{r}_e + \mathbf{r}_f)/2$. With the new result of the root node as the target point, execute a single chain algorithm on all sub chains to complete the IK problem solution.

C. Closed-Loop

Under closed-loop conditions, the core idea of the algorithm proposed in this article is to treat the closed loop as a rigid body while ensuring the accuracy of the algorithm. By finding the direction of motion of the center of mass, the motion of the rigid body is determined, and the motion states of other points on the closed loop are solved to complete the solution of the IK problem. The schematic diagram of the algorithm under closed-loop conditions is shown in Figure 3: where the triangle where O is located is the initial position of the closed-loop, and the triangle where O' is located is the position of the closed-loop after motion.

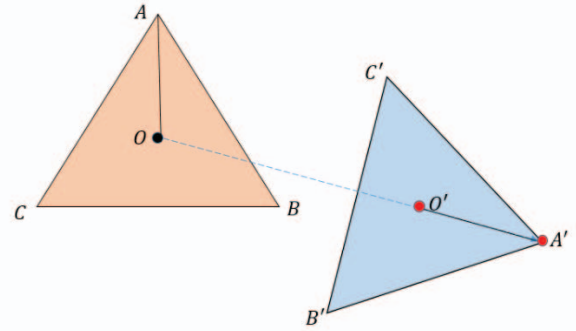


Figure 3 Schematic diagram of NZFIK algorithm under closed-loop conditions

The algorithm requires a given number of related nodes n , a set of joint point positions $\mathbf{P} = \{\mathbf{P}_i, i = 1, 2, 3, \dots, n\}$, a target point position $\mathbf{T}$, and a set of length $L = \{D_i = |\mathbf{P}_i - \mathbf{P}_{i+1}|, i = 1, 2, 3, \dots, n-1; D_n = |\mathbf{P}_n - \mathbf{P}_1|, i = n\}$ between connected joints. The algorithm process is as follows:

Firstly, calculate the centroid coordinate $\mathbf{O} = \frac{\sum_{i=1}^n \mathbf{P}_i}{n}$ of the closed-loop, and then calculate the distance from $\mathbf{O}$ to each joint point within the closed-loop $w = \{w_i = |\mathbf{P}_i - \mathbf{O}|, i = 1, 2, 3, \dots, n\}$. Then, based on the transformations before and after $\mathbf{O}$ and $\mathbf{P}'_n$, calculate the transformation matrix $\mathbf{T}(\mathbf{p})$. Finally, use the transformation matrix $\mathbf{T}(\mathbf{p})$ to calculate the position of the other joint points in the closed-loop to obtain $\mathbf{P} = \{\mathbf{P}_i, i = 1, 2, 3, \dots, n-1\}$.

IV. RESULTS AND ANALYSIS

Both the algorithm in this article and FABRIK are improved algorithms based on the FTL algorithm. Therefore, a comparative analysis was conducted on the three-dimensional display effect and algorithm performance of the algorithm with FABRIK algorithm under single end effector, multi end effector, closed-loop, and human bone model conditions. The computer configuration used in the experimental environment of this article is shown in Table 1.

Table 1 The testing environment of the NZFIK algorithm in this article

CPU	Intel(R) Core(TM) i7-7700K
Number of Cores	8
Frequency	4.20GHz
GPU	NVIDIA GeForce GRX 1080
Operation System	Windows 10 Professional
RAM	16.0GB
System Type	64 bit operation system

The number of iterations and execution time of the FABRIK algorithm strongly relate to the accuracy of FABRIK. The accuracy of the FABRIK algorithm in this article is described using error length/joint length, as described in the following text_H. FABRIK_T. FABRIK_T. The FABRIK algorithm with errors of 1%, 0.1%, and 0.01% for the TT algorithm.

A. Single End-effector

The single chain model used in this article uses FABRIK with a given target point. The display effects of the H algorithm and NZFIK algorithm are shown in Figure 4.

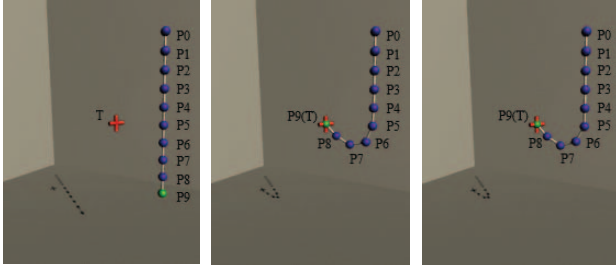


Figure 4 a) Initial situation of single chain condition b) Performance effect of FABRIK algorithm c) Performance effect of NZFIK algorithm

From Figure 4, it can be seen that the performance results of the NZFIK algorithm are similar to those of the FABRIK algorithm, and the actual displacement length of each node is shown in Table 2. From Table 2, it can be observed that the degree of deviation between the NZFIK algorithm and the FABRIK algorithm is almost the same. However, the FABRIK algorithm has some meaningless deviations when dealing with the four nodes P1, P2, P3, and P4, which is not conducive to the stability of mechanical control but also increases the cost of mechanical control.

Table 2 Displacement lengths of each node using different algorithms under single chain conditions

Joint	FABRIK	NZFIK
P0	0	0
P1	0.000002214655	0
P2	0.0001887252	0
P3	0.001585879	0
P4	0.01331015	0
P5	0.1108559	0

P6	0.8487726	0.9767193
P7	3.256737	3.264573
P8	6.483648	6.485281
P9	9.998027	10

Figure 5 compares the algorithm performance of NZFIK algorithm and FABRIK algorithm with different precision to solve Inverse kinematics problems when the single end actuator has 10 joints. It can be seen from the figure that NZFIK algorithm is far superior to FABRIK algorithm in execution efficiency, which is about 3 times the efficiency of FABRIK Algorithmic efficiency with an error of 1%.

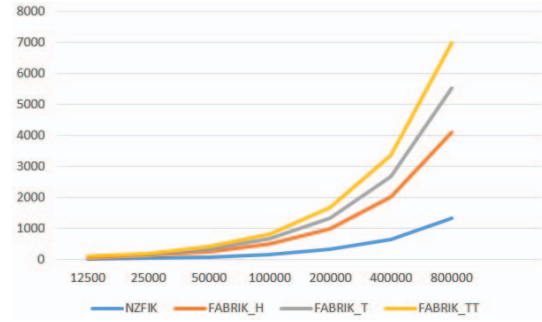


Figure 5 The horizontal axis represents the number of executions, while the vertical axis represents the execution time described in C++exe. (Unit: ms)

B. Multi End-effector

The performance of the multi chain model used in this article, which uses the FABRIK_H algorithm and NZFIK algorithm under a given target point, is shown in Figure 6.

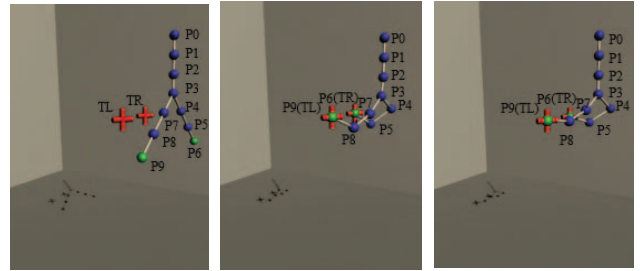


Figure 6 a) represents the initial situation of the multi chain condition, b) represents the performance of the FABRIK algorithm, and c) represents the performance of the NZFIK algorithm

From Figure 6, it can be seen that the performance results of the NZFIK algorithm are similar to those of the FABRIK algorithm, and the actual displacement length of each node is shown in Table 3. From Table 3, it can be observed that the degree of deviation between the NZFIK algorithm and the FABRIK algorithm is almost the same. However, the FABRIK algorithm has added some meaningless deviations to the six nodes P0, P1, P2, P3, P4, and P7, which is not conducive to the stability of mechanical control but also increases the cost of mechanical control.

Table 3 Displacement lengths of each node using different algorithms under multi chain conditions

Joint	FABRIK	NVFIK
P0	0.00027	0
P1	0.000272	0
P2	0.000272	0
P3	0.000385	0
P4	0.006641285	0
P5	0.3325681	0.3390376
P6	4.685264	4.690416
P7	0.04985064	0
P8	0.631908	0.6390126
P9	4.125248	4.123106

Figure 7 compares the algorithm performance of NZFIK algorithm and FABRIK algorithm with different precision to solve the Inverse kinematics problem when the multi end actuator has 9 sub chains, 1 main chain, 10 joint points on each chain, and a total of 91 joint points. It can be seen from the figure that NZFIK algorithm is far superior to FABRIK algorithm in execution efficiency, which is about 3 times the efficiency of FABRIK Algorithmic efficiency with an error of 1%.

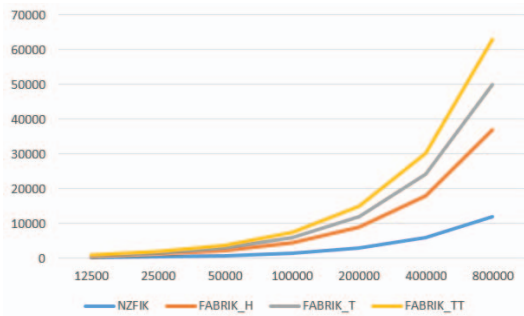


Figure 7 The horizontal axis represents the number of executions, while the vertical axis represents the execution time described in C++exe. (Unit: ms)

C. Closed Loop

The closed-loop model used in this article, as well as the performance of the FABRIK_H algorithm and NZFIK algorithm under a given target point, are shown in Figure 8.

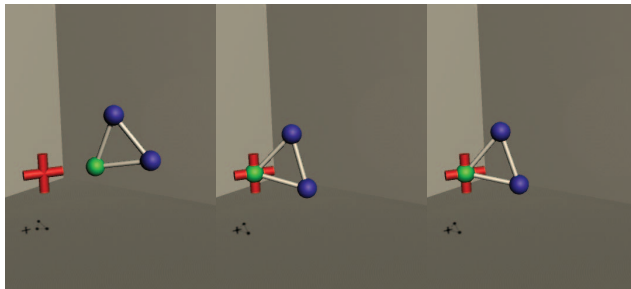


Figure 8 a) represents the initial condition of the closed-loop condition, b) represents the performance of the FABRIK algorithm, and c) represents the performance of the NVFIK algorithm

From Figure 8, it can be seen that the performance results of the NZFIK algorithm are similar to those of the FABRIK

algorithm, and the actual displacement length of each node is shown in Table 4. From Table 4, it can be observed that the degree of deviation between the NZFIK algorithm and the FABRIK algorithm is almost the same.

Table 4 Displacement length of each node using different algorithms under closed-loop conditions

Joint	FABRIK	NVFIK
P0	3.607407	3.689367
P1	1.741249	1.760507
P2	4.472136	4.472136

Figure 9 compares the algorithm performance of NZFIK algorithm and FABRIK algorithm with different precision in solving Inverse kinematics problems when the number of closed-loop joints is 3. It can be seen from the figure that the implementation efficiency of the algorithm proposed in this paper is far better than that of the FABRIK algorithm, which is about 9 times the efficiency of the FABRIK Algorithmic efficiency with an error of 1%.

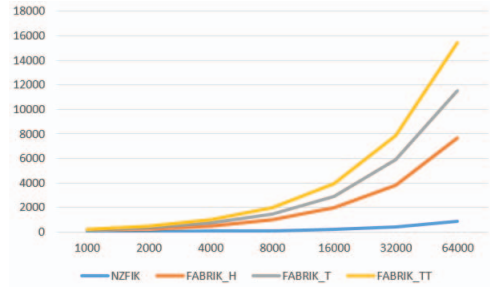


Figure 9 The horizontal axis represents the algorithm type, and the vertical axis represents the execution time described in C++exe. (Unit: ms)

D. Upper Body Skeleton of the Human Body

This article designs a human upper body skeleton model with 9 unconstrained joints and 3 end effectors, which mainly consists of three end effectors: head and two hands. This model includes two closed-loop and five single chains, supporting the testing of the chain algorithm and closed-loop algorithm in the algorithm described in this article. The human skeleton model used in this article is shown in Figure 10.

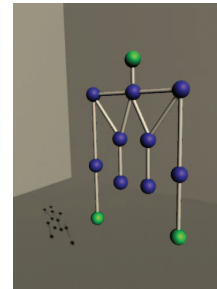


Figure 10 Schematic diagram of upper body skeleton model

The performance of the model using the FABRIK_H algorithm and NZFIK algorithm under a given target point is shown in Figure 11. The target point is completely unconstrained, and there is no limit to the allowed range of

movement for each joint. From Figure 11, it can be seen that the performance of the algorithm proposed in this article is similar to that of the FABRIK algorithm.

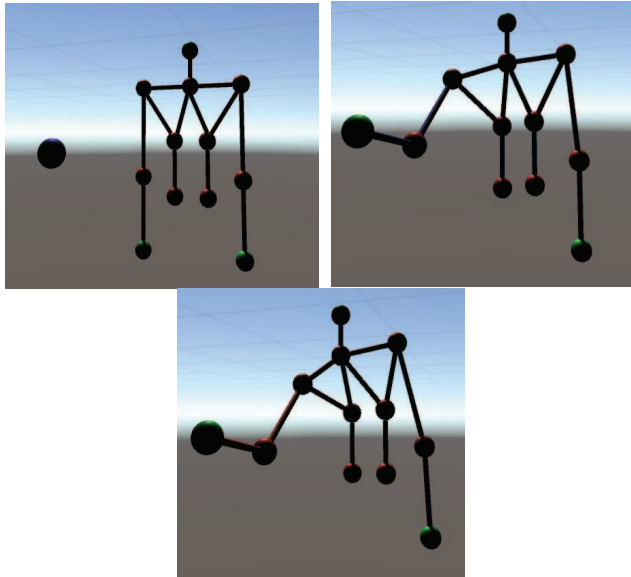


Figure 11 Performance of Different Algorithms under the Same Initial Conditions

Figure 12 compares the algorithm performance of the NZFIK algorithm under the human skeleton condition in Figure 10 with the FABRIK algorithm under different precision to solve the Inverse kinematics problem. It can be seen from the figure that the algorithm proposed in this paper is far superior to the FABRIK algorithm in execution efficiency, which is about 8.8 times the efficiency of the FABRIK Algorithmic efficiency with an error of 1%.

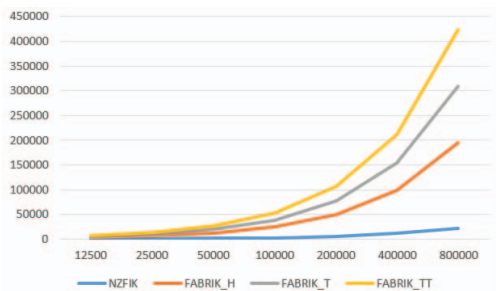


Figure 12 The horizontal axis represents the number of executions, while the vertical axis represents the execution time described in C++exe. (Unit: ms)

V. CONCLUSION

The algorithm proposed in this article mainly targets fine motion and multi node motion situations. For fine motion, the NZFIK algorithm first overcomes the problem of closed-loop distortion when the number of closed-loop nodes is greater than 3, while ensuring the rigid body characteristics of the closed-loop; Secondly, it abandons iterative operations and solves the problem of unavoidable motion errors in iterative algorithms. For multi-node motion, this paper overcomes the problems of the FABRIK algorithm, such as low iteration efficiency and

non-conformance with the laws of kinematics, as the number of nodes in the chain increases, and ensures that the algorithm's operation efficiency increases linearly $O(n)$. The algorithm in this article ensures computational efficiency while making the motion trajectory appear realistic and natural. Subsequent work will add different constraint conditions to the algorithm for real application scenarios, in order to further improve the algorithm and achieve the expected motion return in various scenarios.

REFERENCES

- [1] U. K. G, S. V, N. Ch, G. C. B and Y. K. K, "Estimating 3D Human Pose using Point Based Pose Estimation and Single Stage Method," 2022 3rd International Conference on Computing, Analytics and Networks (ICAN), Rajpura, Punjab, India, 2022, pp. 1-5, doi: 10.1109/ICAN56228.2022.10007130.
- [2] M. S. Allahkaram, M. Javad Ahmadi and H. D. Taghirad, "Closed-form Inverse kinematics Equations of a Robotic Finger Mechanism," 2021 9th RSI International Conference on Robotics and Mechatronics (ICRoM), Tehran, Iran, Islamic Republic of, 2021, pp. 157-162, doi: 10.1109/ICRoM54204.2021.9663448.
- [3] ARISTIDOU A, LASENBY J. FABRIK: A fast, iterative solver for the Inverse Kinematics problem[J/OL]. Graphical Models, 2011, 73(5): 243-260. DOI: 10.1016/j.gmod.2011.05.003.
- [4] L. A. O. Moreno and J. H. A. Alcántara, "An adaptation of FABRIK algorithm for serial robot's inverse kinematics," 2022 IEEE International Conference on Engineering Veracruz (ICEV), Boca del Río. Veracruz, Mexico, 2022, pp. 1-6, doi: 10.1109/ICEV56253.2022.9959663.
- [5] MOYA S, COLLOU D F. A fast geometrically-driven prioritized inverse kinematics solver[C]//Proceedings of the XXIV Congress of the International Society of Biomechanics (ISB 2013). 2013: 1-3.
- [6] ARISTIDOU A, CHRYSANTHOU Y, LASENBY J. Extending FABRIK with model constraints[J/OL]. Computer Animation and Virtual Worlds, 2016, 27(1): 35-57. DOI:10.1002/cav.1630.
- [7] AGRAWAL S, VAN DE PANNE M. Task-based locomotion[J]. ACM Transactions on Graphics (TOG), 2016, 35(4): 1-11.
- [8] TAO S, YANG Y. Collision-free motion planning of a virtual arm based on the FABRIK algorithm[J/OL]. Robotica, 2017, 35(6): 1431-1450. DOI:10.1017/S0263574716000205.
- [9] LANSLEY A, VAMPLEW P, SMITH P, etc. Caliko: An Inverse Kinematics Software Library Implementation of the FABRIK Algorithm[J/OL]. Journal of Open Research Software, 2016, 4(1): 36. DOI:10.5334/jors.116.